SECRETS / LOCAL SANDBOX AND ROLLOUT

MyScoutee Secrets

A hands-on OpenBao introduction and a case-by-case path from scattered local credentials to controlled delivery.

Document	MSC-SEC-VAULT-001 / version 0.1.0 / English
Owner	myscoutee-secret
Audience	Developers, operators, and the product owner
Current state	Disposable OpenBao sandbox works with synthetic data. No production vault or application migration is in place.

The basic idea

A secret manager is a service that stores sensitive values behind an authenticated API. Applications or deployment tools authenticate, read only permitted paths, and pass values to the existing runtime. The web UI is another client of the same service.

Part	What it does here
OpenBao server	Stores versioned sample values in the sandbox. A future persistent server would need storage, TLS, backup, audit, and recovery.
Identity and policy	Decide who can read or change a named path. The sandbox has a powerful development token; real clients need narrower identities.
Delivery path	The current Compose file, Helm chart, or CI variable remains the consumer boundary until its specific integration is proven.

What a vault does not solve by itself

Git-ignored copies, old archives, logs, Terraform state, and issued credentials remain where they are until reviewed. Moving a value does not revoke an earlier copy. The issuer must rotate an exposed credential.

Try the sandbox

This isolated exercise uses a pinned OpenBao image, host loopback port 8200, an ignored local token, and in-memory storage. It never uses a real MyScoutee credential.

From the myscoutee-secret repository

```
./scripts/lab.sh start
./scripts/lab.sh smoke
./scripts/lab.sh token   # use this token in the web login
```

Step	What you should see
1. Open the UI	Visit http://127.0.0.1:8200/ui/ in your local browser. Choose Token login and enter the token printed by the command above.
2. Read the sample	Open Secrets > secret > demo > sample. The example field has the synthetic value sandbox-only-value.
3. Explore	Edit that synthetic value, save it, and inspect its KV version history. You can create another synthetic path.
4. Stop	Run ./scripts/lab.sh stop. The sample disappears because the dev server stores everything in memory.

What the smoke test checks

It verifies server health, the UI route, a KV v2 write and read, and denial of a read with an invalid token. This is evidence that the sandbox works; it does not test a durable store, production TLS, role-based access, backup, or an application connection.

Sandbox boundary

The listener uses HTTP only on 127.0.0.1. OpenBao's development mode starts unsealed and loses data at restart. Never enter a real key, password, token, or customer data in this UI.

Choose the right integration

MyScoutee already has delivery mechanisms. Introduce a vault at their input boundary instead of teaching every application a separate secret API at once.

Case	Existing path	Candidate first step
Developer machine	Ignored local .env.secrets and Firebase files; versioned examples/defaults.	Use an authenticated local fetch to populate the same paths, then check permissions and missing-value failure.
Compose deployment	Package-provided .env.secrets plus a mounted Firebase file.	Render those files on the host before Compose starts. Prove atomic writes, restart, rotation, and rollback.
Kubernetes lab	Helm creates Kubernetes Secret objects from locally supplied credentials.	Compare an agent, CSI, operator, or pre-deploy sync against the current cluster and identity model. Check Helm and etcd copies.
CI laboratory	Ignored inputs become scoped GitLab CI variables.	Keep current variable delivery initially. Later, give trusted jobs a short-lived, narrowly scoped fetch identity.
Operator keys and state	Signing/SSH/TLS keys, kubeconfig, Terraform state, and backups have different owners.	Define custody, signing, retention, and recovery per type. Do not bulk import state into KV.

Backend and template

A shared change to application delivery must be carried into both repositories and verified in both. Their deployment settings and generated image identities remain separate.

OpenBao or HashiCorp Vault?

They are alternative server products. This sandbox uses OpenBao to learn the UI and KV model. Select the long-lived product only after the required authentication, operations, support, and deployment model are clear.

Roll out one credential at a time

Gate	Required evidence before moving on
1. Inventory	Name the credential's issuer, owner, consumers, environment, current path, rotation method, expiry, and recovery owner. Record no value in Git.
2. Persistent store	Configure TLS, scoped authentication, policies, audit, durable storage, backup, a restore exercise, and unseal/key custody. Keep this separate from the sandbox.
3. First consumer	Choose one low-impact static credential. Adapt its existing delivery path and verify startup, restart, missing-vault behavior, access denial, rotation, and rollback.
4. Expand	Repeat per environment and consumer. Compare behavior with the old path before removing old copies.
5. Retire copies	Review Git history, local archives, CI variables, logs, and generated artifacts. Rotate exposed credentials at their issuer; preserve needed recovery evidence.

Questions that must have owners

Who can unseal and restore the store? How do workloads authenticate after restart? What happens when the store is unavailable? Where do fetched values land on disk, in Kubernetes, or in CI logs? Who rotates a credential and confirms every consumer has the new version?

Source and limits

This guide reflects the repository state on 2026-10-29 and the disposable sandbox. The local path inventory is Git-ignored. Neither the sandbox nor this guide proves that history, generated artifacts, or operational backups are free of secrets.

Primary documentation

OpenBao: UI, KV v2, integrated storage, development mode. HashiCorp Vault: production hardening, Kubernetes integration comparison.